HW 6

ENGR 21, Fall 2026.

October 22, 2026

Due Date	Thu, Oct 22
Turn in link	Gradescope PDF and Code
URL	`emadmasroor.github.io/E21-F26/Homework/HW6`

What to turn in

Combine all written HW into one PDF file.

Problem	Format
1	Code: One `.py` file containing all TFWR code.
2	PDF
3	PDF
4	PDF
5	Code: One `.py` file **and** PDF with state transition diagram

1 The Farmer Was Replaced

Download the save file here to ensure that you've unlocked all that's required for this assignment.



Figure 1: Look how much stuff you'll have!

The Farmer Was Replaced now has 'Polyculture favorites'. To check if a particular plant has a 'Polyculture favorite', use the function `get_companion()` while your drone is hovering over a cell with a plant growing on it. The function `get_companion()` returns two outputs. The first output is the type of plant that would be this plant's 'Polyculture favorite', and the second output is a tuple of x-y values that tell you where this companion plant should go.

1.1 A function to navigate to x,y

Write a function that accepts as argument a tuple of `(x,y)` values. When this function is called, your drone should navigate to the spot `(x,y)`.

1.2 A function to plant companions

Write a function that

1. Checks if a plant has a companion. If not, it doesn't do anything.
2. If it does have a companion, the drone navigates to the companion's favored location. You should make use of the function you wrote in Section 1.1.
3. Once it is at the favored location, it plants the desired companion plant at this location.
4. The drone then returns to the location of the first plant.

The above should work regardless of whether there is currently a plant at that location. Assume that the entire farm is tilled soil, not grassland, for this purpose.

2 Two-state FSM



Figure 2

2.1 Four ways of representing a Finite State Machine

A coin-operated turnstile, shown in Figure 2, is to be designed using a two-state finite-state machine. It has two states: Locked and Unlocked. There are two possible inputs that it can accept: (1) you can put a coin in it, and (2) you can push through the turnstile.

The turnstile operates as follows.

If the user enters a coin while the turnstile is locked, the turnstile becomes unlocked. Adding any more coins doesn't do anything; the turnstile remains unlocked. If the user pushes through the turnstile while it is unlocked, the turnstile lets him through and the turnstile locks again. If the

user tries to push through the turnstile while it is locked, nothing happens and the turnstile stays locked.

Using the given states and inputs,

1. Construct a state-transition diagram. You are encouraged to make use of the 'Finite State Machine Designer' website here to produce professional quality diagrams, but this is not necessary. Label your states and inputs descriptively with words, rather than numbers.

2. Make a state-transition table. Use words, not numbers, to label your states and inputs.

3. Make a truth table using the following values as 'names' in binary:

- the unlocked state is `0`,
- the locked state is `1`,
- adding a coin is encoded by `0`
- pushing the turnstile is encoded by `1`.

4. Make a logic diagram for this truth table using And, Or, and Not gates.

3 Modified Finite State Machine

You are tasked with designing a new finite state machine that allows up to three people at a time to go through the turnstile at a cost of $0.50 each. The machine only accepts quarters. When two quarters have been entered into the turnstile, one person can pass through before the machine becomes locked again.. If four quarters are entered into the turnstile, two people can pass through before it becomes locked again. Entering more than $1.50 does not have any effect. So, for example, if $2.00 is entered into the machine, it will still only let three people pass, not four.

1. Draw the state transition diagram for this finite state machine, labeling each state with the number of cents/dollars that have been entered. Assume that the initial state is $0.00.

2. How many rows and columns will the state-transition table for this FSM have? You do not need to write it out.

3. Write out the truth table for this finite state machine. Each state should be labeled by the number of quarters it currently recognizes inside the machine, in binary. Entering a coin should be labeled `1` and pushing through should be labeled `0`.

4 Logic Gates and Boolean expressions

4.1 Equivalence of Boolean expressions

You may wish to consult this trurth table generator for this assignment.

The following truth table expresses the relationship between 3 'inputs' a, b and c, and one 'output' d (this is why there are $2^3 = 8$ entries). Here, the word 'input' could refer to either a FSM's input or a FSM's current state. The word 'output', in a FSM context, refers to the next state of the FSM for that combination of inputs and states.

a	b	c	d
F	F	F	F
F	F	T	F
F	T	F	T
F	T	T	F
T	F	F	T
T	F	T	T
T	T	F	T
T	T	T	T

There are several different ways to express this truth table. For example, the following would work.

- d = (a and b and c) or (a and b and not c) or (a and not b and c) or (a and not b and not c) or (not a and b and not c)

This is an unnecessarily verbose way of describing this truth table. There is a much simpler expression that involves only **three** boolean operators when written out in English. Find a simpler expression of this truth table using at most three boolean operators.

> **i** Note
>
> The expression 'a and b and c' is an example of a Boolean expression with two operators. The expression '(a and not b) or c' has three operators.

4.2 Simplifying Truth Tables

The following truth table can be expressed in the following way: c = (not a and not b) or (not a and b). However, there is a much simpler Boolean expression that can be used here instead to relate c, the output, to a and b, the inputs. Find the most simple statement of how c depends on a and b. Then, draw the logic diagram for both the original statement c = (not a and not b) or (not a and b) and for your simplified statement.

a	b	c
F	F	T
F	T	T
T	F	F
T	T	F

5 A new combination lock

Your task in this part of the homework assignment is to create a 'combination lock', similar to the one discussed in class, that 'unlocks' when the code `ABABBABA` is entered. The 'locked' state is shown

with eight consecutive neopixels shown red; each correct entry (A or B) should sequentially turn the neopixels green, one by one. You do not need any states after the fully-unlocked state is reached.

1. Draw the State Transition Diagram for this system. Please use S_1, S_2, … for your state names, and A/B for the input names.
2. Implement this on the Circuit Playground Express, similar to the code used in class.